PolyGlot

A compiler for mixed syntax

Shared semantics. Typed IR. Custom bytecode.

Compiler Design Project

The problem and our approach

Familiar syntax can share a common compiler implementation

The problem

Different syntax usually means different tools and runtimes.

Our approach

Normalize accepted constructs into one internal language.

The project studies syntax normalization and a complete compiler pipeline.

One program, several syntax families

Functions and variables interact through the same symbol table

```
int add(int a, int b) {
    return a + b;
}

def twice(value: int) -> int:
    return add(value, value)

LET base = 3
console.log(twice(base));
```

Output

```
6
```

C-style function
Python-style function
BASIC assignment
JavaScript output

Language model and scope

A statically checked, imperative and procedural language

Structured control flow

Nested if/else, loops, break and continue.
Braces and indentation can coexist.

Functions and storage

Typed functions, forward calls and recursion.
Global bindings and frame-local storage.

Shared values

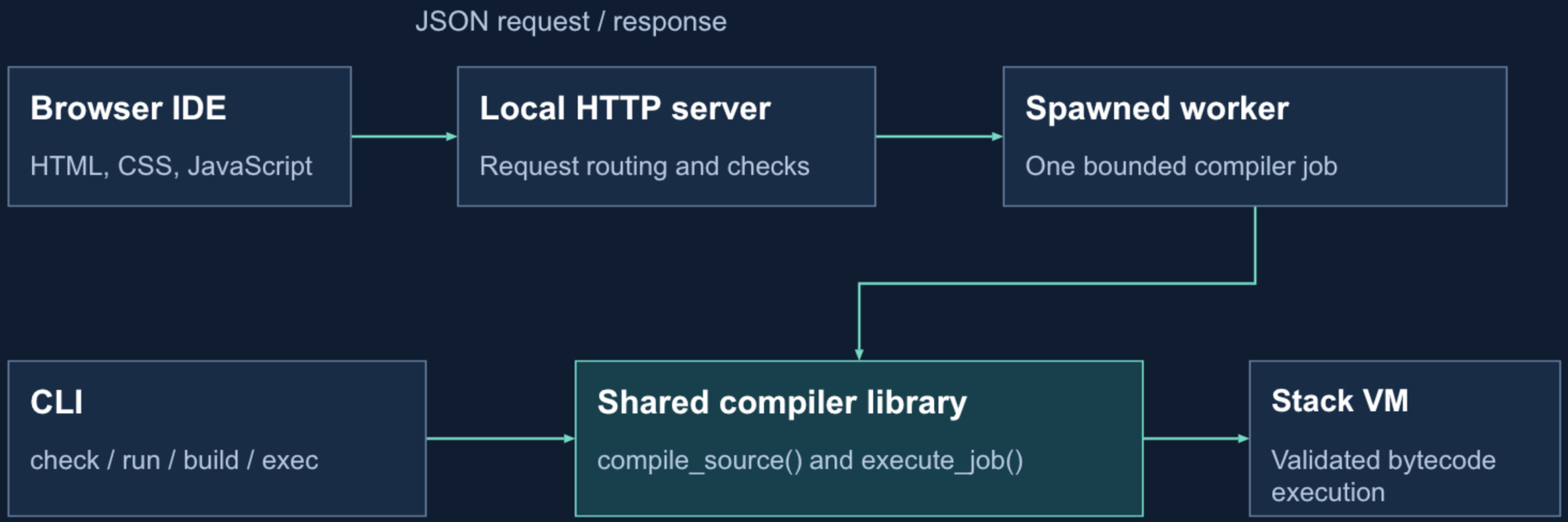
int, float, bool, string and fixed-length one-dimensional arrays.

Accepted syntax

Selected forms inspired by Python, C, C++, Java, JavaScript and BASIC.

System architecture

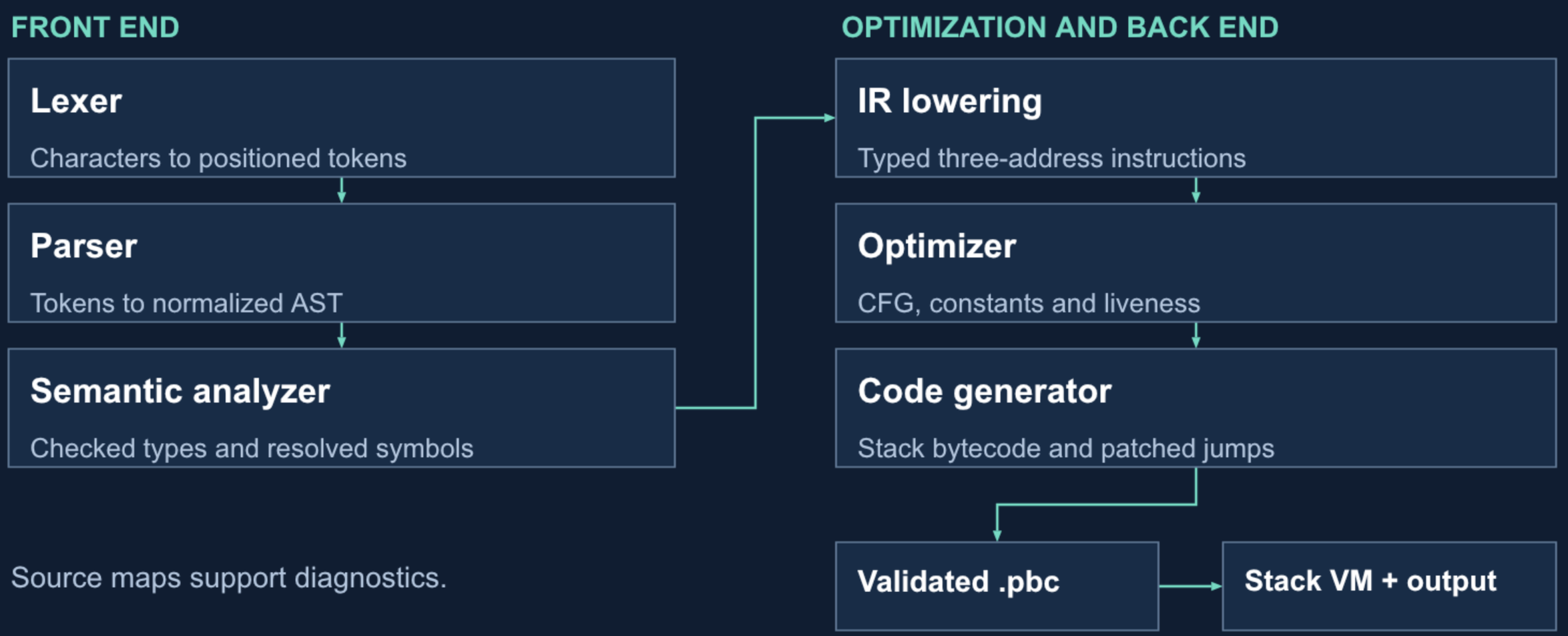
The IDE and CLI use the same compiler library



Compilation returns stage data and diagnostics. Run additionally executes the artifact.

Compiler architecture

Representations connect every stage of compilation



Source maps support diagnostics.

Syntax recognition inside the parser

Leading tokens and bounded lookahead guide grammar dispatch



All paths use shared expression rules and create common AST nodes.

Lexical analysis

The scanner converts characters into positioned tokens

```
int result = 2 + 3;
```

Token text	Token category	Purpose
int, result	NAME	Parser interprets names and keywords
2, 3	NUMBER	Numeric literal values
=, +, ;	OP	Assignment, addition and separator
newline, end	Layout / end markers	Statement boundaries and termination

Each token retains a source span for diagnostics and editor navigation.

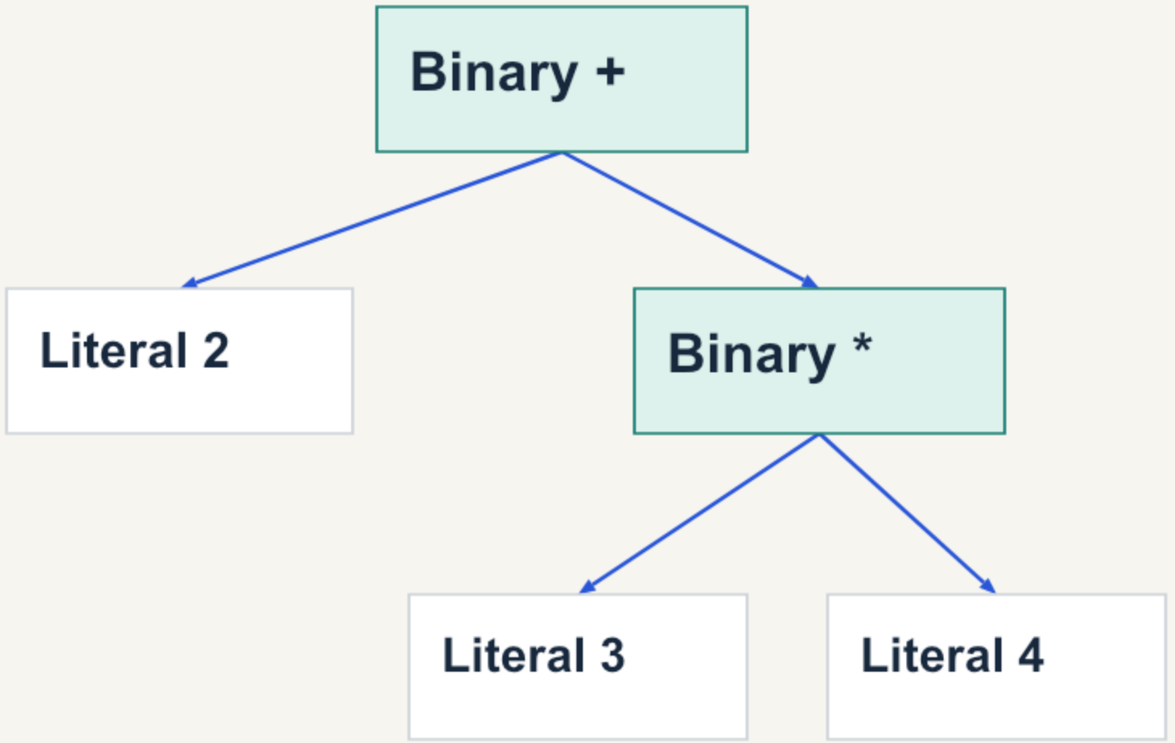
Parsing and operator precedence

Recursive descent handles statements. Pratt parsing handles expressions.

```
print(2 + 3 * 4)
```

Binding power

Multiplication binds more tightly than addition. Parentheses can change the grouping.



Expression subtree shown. This program prints 14.

AST normalization

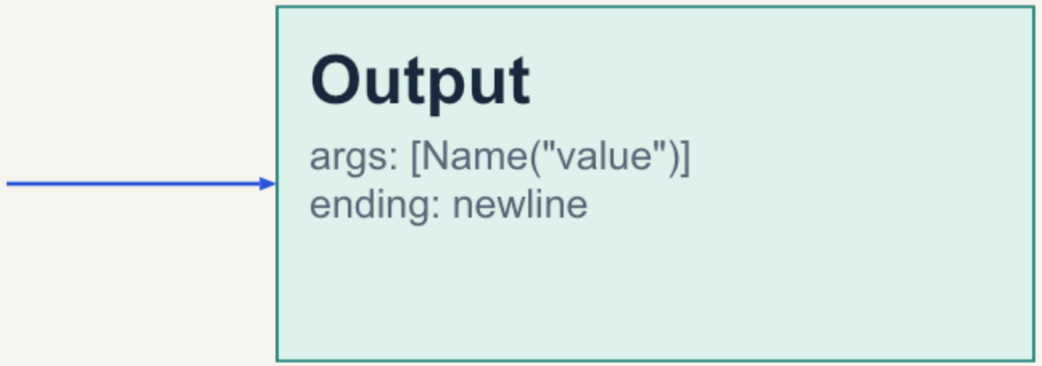
Equivalent output spellings become the same semantic node

```
print(value)

console.log(value);

System.out.println(value);

PRINT value
```



One AST node kind

Restricted printf additionally lowers explicit formatting operations.

Structured blocks across syntax families

The parser tracks suites in their surrounding block context

```
int score = 0;

for (int i = 0; i < 4; i++) {
    if i % 2 == 0:
        score += i
    else:
        score += 10
}

print(score)
```

C-style outer loop

Braces delimit the loop body.

Inner branch

Indentation delimits each suite.

examples/mixed_nesting.poly prints 22.

Semantic analysis and the symbol table

Parsing checks structure. Semantic analysis checks meaning.

Name resolution and type checking annotate the AST before IR generation.

Binding	Checked type	Role	Storage
bonus	int	Immutable binding	global slot 0
values	int[]	Array variable	global slot 1
sum_values	(int[]) to int	Function	function ID 1
items	int[]	Parameter	local slot 0
total	int	Local variable	local slot 1

Selected actual symbols from p1_mixed.poly. The analyzer collects function signatures first.

Errors stop at the failing phase

Earlier stage data remains available for inspection

Source example	Diagnostic	Compilation result
print(2 +)	P010: Expected an expression.	Parser stops. No completed AST.
count = 3 count = "three"	S004: Cannot use string where int is required.	Semantic analysis stops. No IR or bytecode.
print(1 / 0)	R004: Division or remainder by zero.	Compilation succeeds. The VM reports a runtime error.

Diagnostics include filename, line, column and a source excerpt.

Typed three-address intermediate code

A common IR removes dependence on the original syntax

```
print((2 + 3) * 4)
```

Explicit computation

Each arithmetic operation defines a typed temporary.

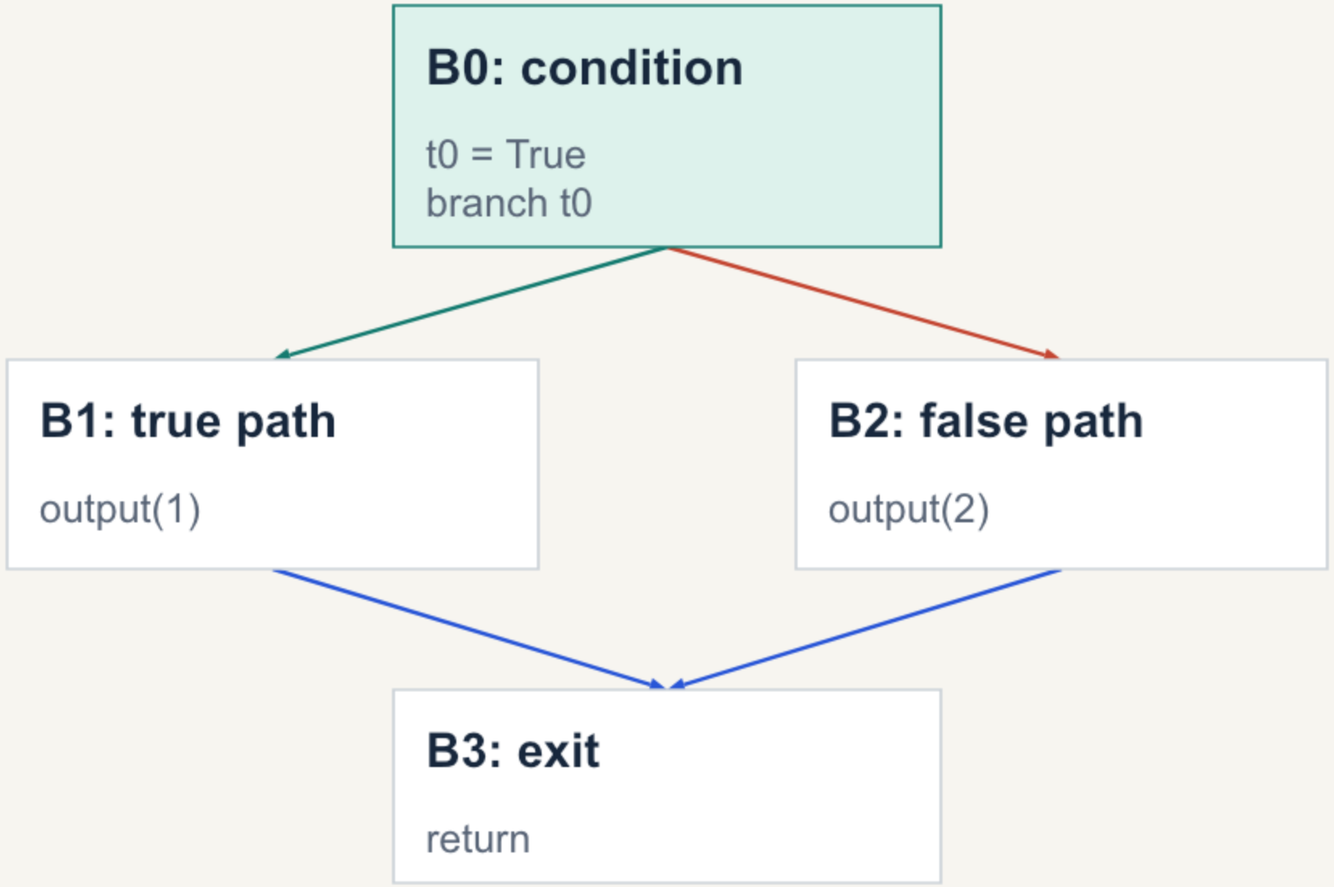
```
t0 = 2          : int
t1 = 3          : int
t2 = t0 + t1    : int
t3 = 4          : int
t4 = t2 * t3    : int
output(t4)
return
```

Readable rendering of the original IR. Every instruction also retains source metadata.

Basic blocks and the control-flow graph

A known branch exposes an unreachable path after simplification

```
if True:
    print(1)
else:
    print(2)
```



Before optimization

Both branch targets exist in the original control-flow graph.

Liveness and next-use analysis

The optimizer needs evidence before deleting a temporary definition

```
OUT[B] = union of IN[S]
         for each successor S

IN[B]  = USE[B] union
         (OUT[B] - DEF[B])
```

Backward fixed point

A worklist revisits affected predecessor blocks.

Next-use analysis

A reverse scan distinguishes overwritten temporary definitions.

The current optimizer uses these facts to remove safe unused CONST definitions.

Implemented IR optimizations

Conservative transformations preserve observable behavior

01 **Local constant folding**

Compute proven primitive operations during compilation.

02 **Constant branch simplification**

Replace a known Boolean branch with a direct jump.

03 **Unreachable block removal**

Discard blocks that have no path from function entry.

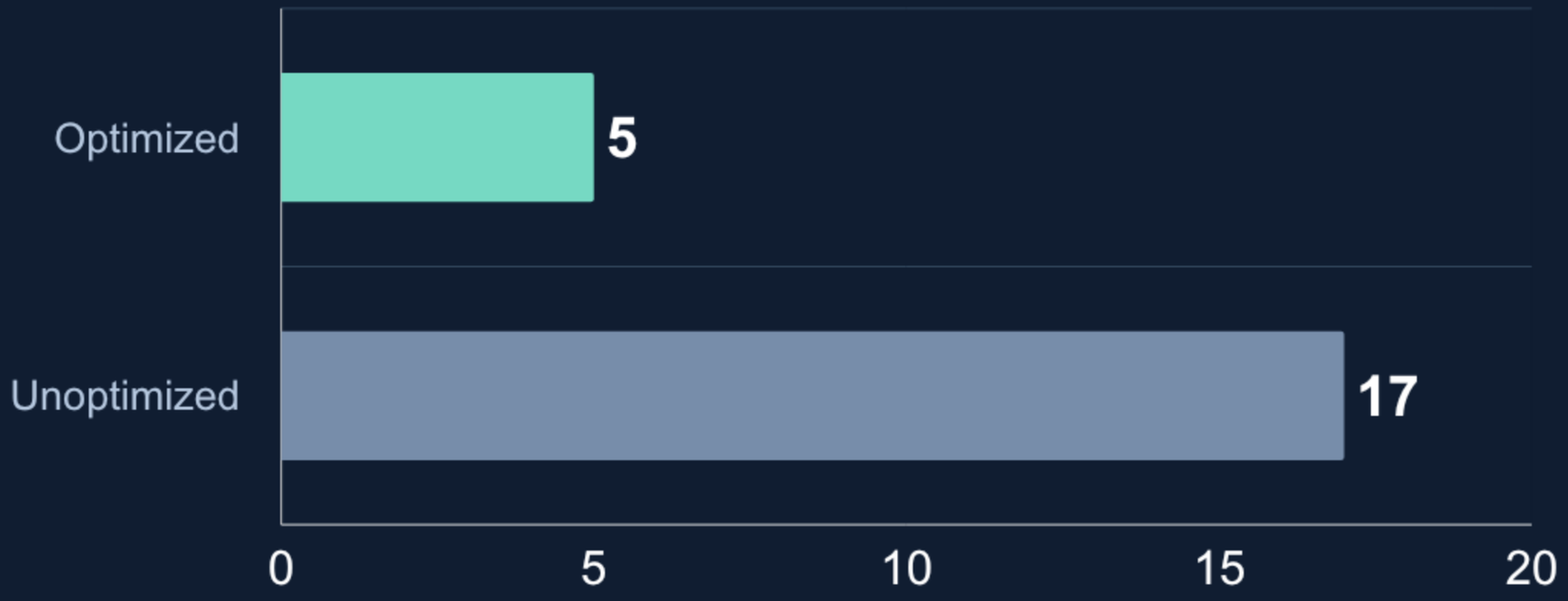
04 **Dead constant removal**

Remove unused CONST definitions using liveness and next use.

Reachable calls, stores, output and potentially failing operations remain.

Optimization result: 17 instructions become 5

print((2 + 3) * 4) produces 20 in both runs



70.6%

fewer executed instructions in this example

Instruction counts measure this example. They do not establish a general timing speedup.

Bytecode generation

The code generator lowers IR into stack-machine instructions

```
t2 = t0 + t1
```

Typed three-address IR

```
LOAD_LOCAL  <slot of t0>
LOAD_LOCAL  <slot of t1>
BINARY      "+"
STORE_LOCAL <slot of t2>
```

Storage allocation

Declared local slots come first.
IR temporaries use additional slots.

Instruction emission

Constants, calls, output and array operations have explicit lowering rules.

Backpatching jump targets

Symbolic labels become absolute instruction indexes

```
BRANCH condition, L_true, L_false

LABEL L_true
    ...

LABEL L_false
    ...
```

Code generation handles both conditional and unconditional jumps.

First pass

Emit instructions and record unresolved jump positions.

Label map

Record the instruction index associated with each label.

Patch pass

Replace each placeholder target with its resolved numeric index.

A reusable bytecode artifact

The .pbc file separates compilation from execution

```
format: "polyglot-bytecode"
version: 3
entry: 0
constants: [...]
global_count: ...
functions: [...]
```

Portable representation

UTF-8 JSON stores opcodes,
operands,
function signatures and source maps.

Validated input

The loader checks structure, targets,
slots and operand-stack consistency.

The VM can execute a valid artifact without reading the original source file.

Execution on the stack VM

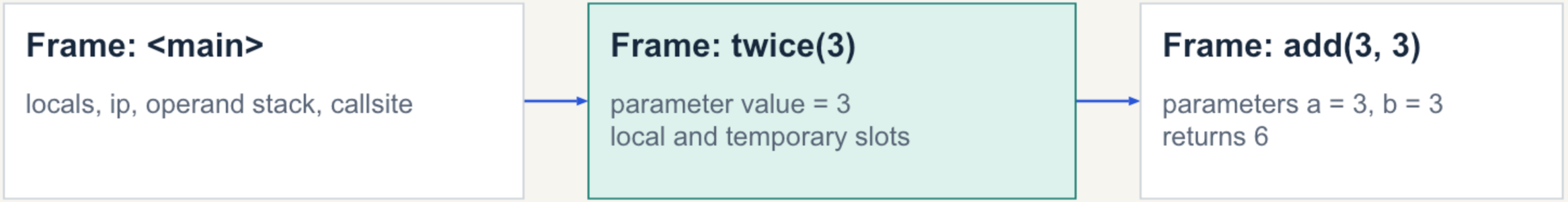
The optimized expression uses one constant and five instructions

Instruction	Operand stack after execution	Effect
CONST 0	[20]	Push constant pool entry 0
STORE_LOCAL 4	[]	Save 20 in temporary slot 4
LOAD_LOCAL 4	[20]	Load the saved value
OUTPUT 1, "", "\n"	[]	Print 20 and a newline
RETURN_VOID	[]	Finish the main function

The VM executes our instruction set and maintains its own instruction pointer.

Function calls and activation records

Each call owns a separate frame



Call sequence

Evaluate arguments, create a frame, then execute the callee.

Return sequence

Restore the caller and push the returned value onto its stack.

Default VM limits: 1,000,000 steps, recursion 64, 64 KiB output and 5 seconds.

Arrays and iteration lowering

A Java-style declaration can feed a Python-style loop

```
int[] arr = {1, 2, 3};

for num in arr:
    print(num)
```

Shared representation

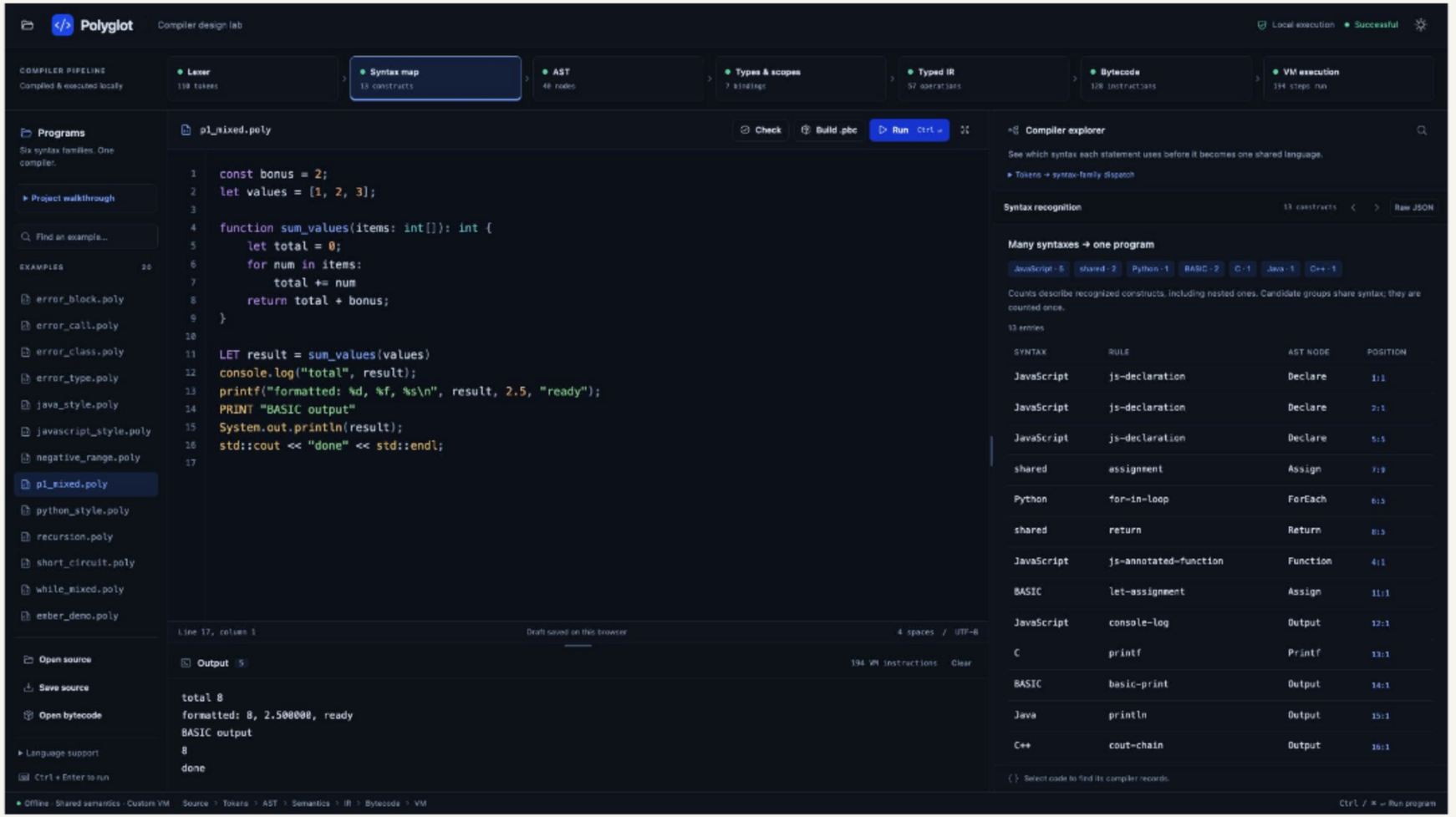
int[] has one fixed element type.
Array references cross function calls.

Loop lowering

Capture the array once, compute length,
and iterate using an internal index.

ARRAY_GET and ARRAY_SET enforce index types, bounds and element types at runtime.

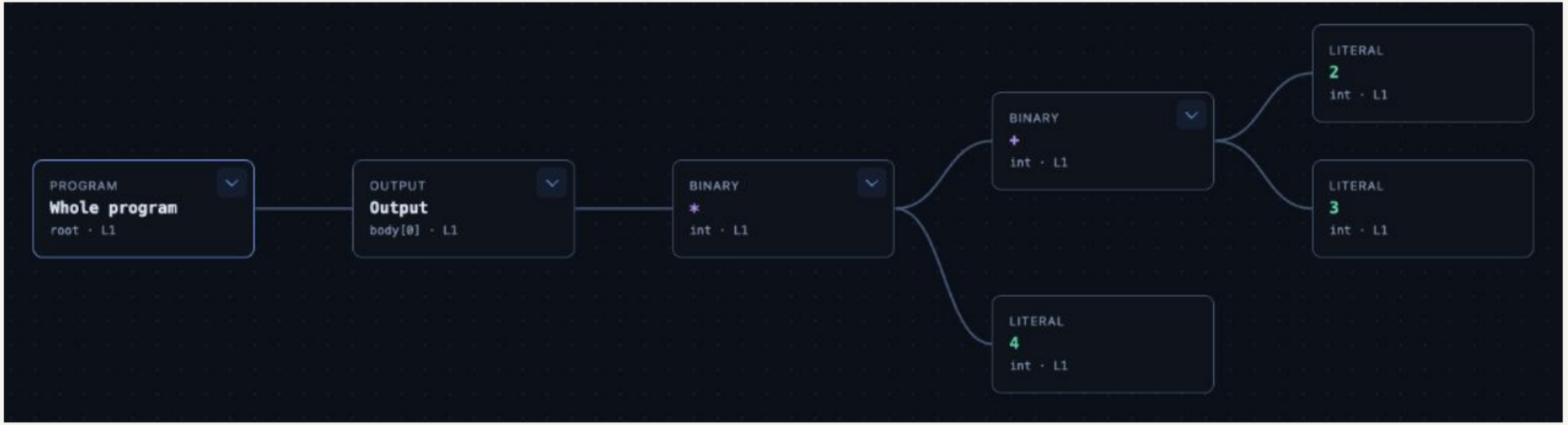
The working compiler workspace



Actual stage representations, mixed source and program output in one workspace.

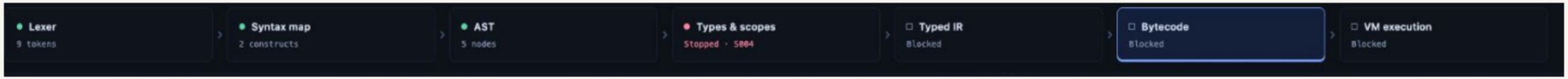
The AST inspector shows semantic structure

print((2 + 3) * 4)



Parent-child edges follow the compiler AST. Checked types and source links stay attached.

Diagnostics in the IDE




```
count = 3
count = "three"
```

Tokens and AST available
Semantic analysis stopped
IR, bytecode and VM blocked

```
Compilation stopped.

error_type.poly:2:1 · S004
Cannot use string where int is required.
count = "three"
```

S004 at line 2: Cannot use string where int is required.

Technology stack and execution modes

The same Python compiler supports two delivery paths

Local application

Browser UI

HTML / CSS / JavaScript

Python HTTP worker

Shared compiler + custom VM

Static browser build

Static website

JavaScript transport and assets

Web Worker + Pyodide

WebAssembly Python runs the same compiler

Pages deployment requires repository-owner configuration. The local IDE needs Python 3.11+.

Verification evidence

Verified implementation snapshot, 26 October 2026

298

Python tests passed

57

Frontend tests passed

Compiler stages, HTTP workers, artifact validation, optimization equivalence and editor interactions

Tests include negative cases and source-independent bytecode execution.

Live demonstration sequence

A short sequence makes each implementation claim observable

Demonstration	What to inspect	Expected result
p1_mixed.poly	Syntax map, symbols and output	Five output lines
mixed_nesting.poly	Nested AST and control flow	22
print((2 + 3) * 4)	Original IR and generated bytecode	20, optimized run: 5 steps
error_type.poly	Diagnostic and blocked phases	S004 on line 2
Build, then execute .pbc	Bytecode artifact without source	Original output repeats

The appendix contains launch commands and suggested presenter responsibilities.

Current boundaries and future work

The project implements a defined subset with explicit limitations

Language boundary

Classes, imports, pointers and multidimensional arrays remain outside scope.

Analysis boundary

One top-down parsing strategy and conservative local constant reasoning.

Next compiler work

Multiple-error recovery and broader data-flow optimization.

Next evaluation work

Larger programs, elapsed-time benchmarks and more grammar-extension tests.

What the project demonstrates

Many accepted syntaxes.
One compiler implementation.

A shared AST and type system
Real IR analysis and optimization
Validated bytecode on our own VM

Appendix: implementation map

Where each architectural responsibility lives

Responsibility	Repository module
Source positions and tokens	source.py, lexer.py, layout.py
Grammar and normalized AST	parser/, ast.py
Types, scopes and bindings	semantics.py, types.py
IR and optimization	ir.py, analysis.py, optimizer.py
Code generation and artifact checks	codegen.py, artifact.py
Execution and user interfaces	vm.py, cli.py, web/server.py, web/worker.py

Appendix: complexity and performance

Multiple syntax families add checks rather than six full compilation passes

Lexing and parsing

Typically approximately linear in source characters and tokens for ordinary input.

Optimization cost

CFG construction and iterative liveness add work based on blocks and edges.

Program complexity

A loop over n values stays O(n).
Two full nested n loops stay $O(n^2)$.

Runtime overhead

The interpreted VM adds per-instruction overhead compared with native code.

Appendix: short viva answers

Is this six complete compilers?

One defined language accepts selected syntax families.

Do we detect a whole-file language?

Grammar dispatch recognizes constructs while parsing.

Who generates the bytecode?

Our codegen.py lowers typed IR into our instruction set.

Is Python executing the submitted source?

Python hosts our compiler and VM. The VM executes bytecode.

Why does mixing syntax work?

Names, types and operations share one semantic model.

Appendix: four-presenter plan

Suggested speaking split with a concrete handoff at each stage

Presenter	Main slides	Responsibility
1	1–7	Problem, scope, system architecture and grammar dispatch
2	8–13	Lexer, parsing, AST, mixed blocks and semantic errors
3	14–20	IR, CFG, data flow, optimization and bytecode generation
4	21–32	Artifacts, VM, arrays, live demo, evidence and boundaries

```
cd /Users/Umang/Documents/polyglot
python3 run_polyglot.py serve --port 8766
```